AI Engineering and Claude Architect · Part IV, Production

AI Production Engineering, Lesson 6

Deployment, Multi-Tenancy and Disaster Recovery

Claude Sonnet 4.5 retires on 30 November 2026. Requests to it will then fail. A model isn't a dependency you upgrade when you feel like it, and a one-line prompt change can break more than a code release.

Module	AI10 of 13
Lesson	6 of 7
Prerequisites	AI10 L1–L5, AI09 L4, HLD M13
Exam relevance	Mostly out of scope. Supports TS 3.6
Reading time	~30 minutes

Contents

1. Deployment, Multi-Tenancy and Disaster Recovery

Lesson 6 of 7 · AI Production Engineering

Deployment, Multi-Tenancy and Disaster Recovery

Key idea

Prompts and models are production code that behaves probabilistically, so release them in stages, with evals and a way back. Give each tenant a fair share, because one noisy customer can queue everyone. Plan recovery around what's really correlated, and test it, because failover moves load onto something that may have no room.

> *Before: reliability, gateways, evals and tenancy controls. Teaches: release stages for prompts and models, model lifecycle, fair sharing, platforms and residency, availability arithmetic, and recovery. Used by: L7.*

The hook: a date that won't move

From Anthropic's model deprecations page, as of today:

Model	State	Retirement
`claude-sonnet-4-5-20250929`	Deprecated on 30 September 2026	**30 November 2026**
Recommended replacement	`claude-sonnet-5-5`	

```
Days from 2026-11-05 to Sonnet 4.5's published retirement: 25
```

The page says requests to models past the retirement date will fail, and that customers with active deployments get at least 60 days' notice for publicly released models.

Now consider what that migration means. It's not just a new string. The 5.x models changed behaviour and parameters: `temperature` was removed in API version `2026-10-21`, thinking settings differ, and forced `tool_choice` returns a 400 on Sonnet 5.5. Output style can change, so every eval needs rerunning, and thresholds need recalibrating (L3).

A team without an eval suite and a release process has 60 days' notice to build both. Build them before the notice arrives.

Intuition: changing an airline's pilots

Imagine a new, very capable pilot joins an airline. You wouldn't hand them the fleet on day one. They'd do simulator hours, fly as a silent observer, then fly a few short routes with supervision, then more. If something looks wrong, you can pull them from the schedule that day.

Simulator, observer, short routes, and the full schedule are evals, shadow, canary and ramp. A new model or prompt is a new pilot. The route's passengers don't get a say, so you take this seriously.

Mechanism

What counts as a release

In an AI system, many things beyond code can change behaviour:

Artifact	Why it needs care
System prompt	One line can change every answer, and break the cache
Model ID and parameters	Different output, and different valid parameters
Tool descriptions	Change selection (TS 2.1)
Retrieval index, chunks, embeddings	Different context (AI05)
Few-shot examples	Shift behaviour
Guardrail thresholds	Change what's blocked (AI09 L5)
API version	Removes parameters
SDK version	Telemetry names, retries

Version them all, in one place, and log them per request. Then "what changed on Tuesday?" has an answer.

The stages

AI01 L2 introduced this ladder. Here's the version for prompts and models:

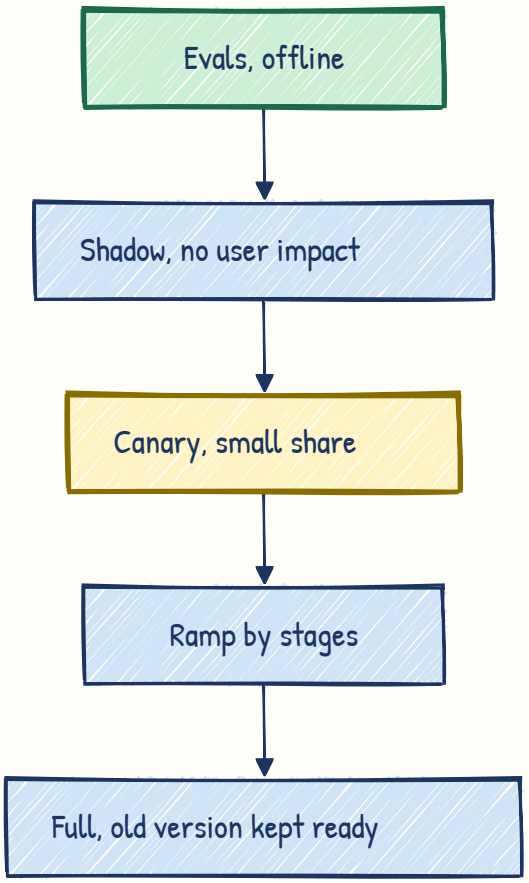


Figure 1. Each stage limits who can be harmed, and each has a way back. Risk grows only as evidence does.

Stage	What runs	What you learn	Cost or limit
Evals	The new version on the suite	Known cases, cheaply	Only what's in the suite
Shadow	New version on real traffic, output discarded	Real inputs, cost, latency, errors	Pays twice. Tools with effects must not run
Canary	A small share of real users	Real outcomes	Some users see problems
Ramp	5%, 25%, 50%, 100%	Load effects	Time
Rollback	One config change	Recovery	Keep the old version available

A shadow can't run write tools. It should use stubs, or skip them. So it's strong for retrieval and answers, and weaker for agents that change state. An agent shadow runs to its first write, unless you build a sandbox.

How long a canary needs

People watch a canary for an hour and call it good. Here's what detection takes, at 95% confidence and 80% power:

```
Requests per arm to detect a drop (95% confidence, 80% power):
  from 95.0%, a drop of 5.0 points: 431
  from 95.0%, a drop of 2.0 points: 2,207
  from 95.0%, a drop of 1.0 points: 8,146
  from 90.0%, a drop of 2.0 points: 3,834
  from 99.0%, a drop of 0.5 points: 7,738
```

So a one-point drop takes about 8,000 **labelled** outcomes in each arm. If you only have signals such as thumbs or escalations, you need many requests per signal. For a low-traffic feature, that may take weeks. Know what your canary can really detect, and use larger evals for the smaller drops.

Canary by tenant or by session, and not by request. A conversation that changes model mid-way can behave strangely, and thinking blocks are bound to their conversation (L1). Also watch for cache effects, as a new prompt starts with a cold cache, so its first costs look worse.

Model lifecycle

The deprecations page defines the states:

State	Meaning
Active	Fully supported, and recommended
Legacy	No more updates, and might be deprecated in future
Deprecated	Still works, with a replacement and a retirement date. The page warns that these are likely to be less reliable
Retired	Requests fail

Partner platforms set their own schedules. The dates apply to Anthropic-operated platforms. Amazon Bedrock and Google-managed Google Cloud have their own, so a fallback on a partner platform may retire on a different date.

Practical habits:

Habit	How
Inventory	The Console's usage export gives usage by API key and model
Calendar	Record each model's date, and alert long before
Alert in production	On any call to a deprecated model
Logical names	One config change (L2)
Rehearse	Migrate a low-risk route first

Multi-tenancy: fair sharing

AI09 L4 covered isolation of data. This is isolation of capacity. One provider limit, shared by three tenants, in a simulation. **A floods at 300 a second for one minute. B and C send 20 a second each.** Real output:

```
One shared limit of 100/s. A floods at 300/s for a minute:
  one shared queue:
    A-flood  served  10600, median wait  42 s, p95 wait  80 s
    B        served    700, median wait  43 s, p95 wait  82 s
    C        served    700, median wait  44 s, p95 wait  82 s
  a queue per tenant, served in turn:
    A-flood  served   7200, median wait  48 s, p95 wait  92 s
    B        served   2400, median wait   0 s, p95 wait   0 s
    C        served   2400, median wait   0 s, p95 wait   0 s
```

With one queue, two innocent tenants wait 43 seconds or more, because of somebody else's spike. Per-tenant queues, served in turn, drop their wait to zero. The flood only slows itself. **Fairness is a scheduling decision, and a first-in-first-out queue doesn't make it for you.**

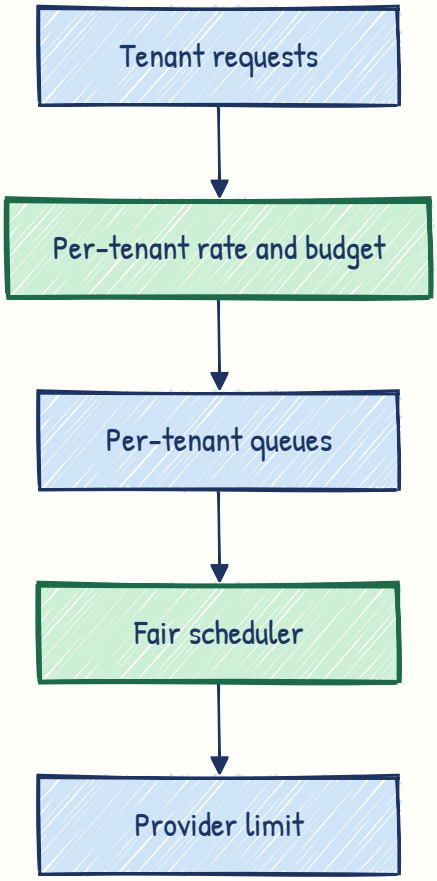


Figure 2. Per-tenant limits stop a flood early. Round-robin shares what's left. The provider's limit sits behind both.

Refinements:

Refinement	Why
Weight by plan	Paid gets more share
Limit by tokens, not requests	A 100,000-token request isn't one unit
Reserved capacity for premium tiers	Some spikes shouldn't touch them
Per-tenant concurrency limits	Long streams hold connections
Dedicated workspace or key for big tenants	Provider-level separation, up to org limits

Tiers of isolation, for capacity:

Level	Isolation	Cost
Shared with fair queues	Good	Low
Separate workspace	Provider-enforced limits	Moderate. Org limit still shared
Separate organization	Strong	Tiers and billing per org. High
Separate platform account	Strongest	Highest

Where the model runs

Claude is available on several platforms, and who processes the data differs:

Platform	Data processor, per the retention page
Claude API, Claude Platform on AWS	Anthropic
Microsoft Foundry, Claude Models on Google Cloud (Anthropic managed)	Anthropic
Amazon Bedrock, Claude Models on Google Cloud (Google managed)	The cloud provider

Data residency: `inference_geo: "us"` requests US-only inference, at 1.1 times the price on Claude 4.6 and later models. On the rate limits page, limits are shared across `inference_geo` values, so this isn't extra capacity.

Compliance is your compliance team's call, as in AI09 L4. Features and limits differ by platform, so read each one's docs. L1 warned that a fallback on another platform needs its own evals.

Availability arithmetic

With **illustrative figures**:

```
Availability, illustrative figures:
  provider 99.90%, gateway 99.95%, vector DB 99.90% in series: 99.750%
  two providers, independent: 99.99990%
  two providers, 20% of outages shared: 99.9799%
  99.00% is 5,256 minutes down a year
  99.90% is 526 minutes down a year
  99.99% is 53 minutes down a year
```

Three things stand out:

1. **Serial dependencies multiply downward.** A RAG agent depends on more than the model.
2. **Independence is an assumption.** Sharing 20% of outages makes the number about 200 times worse, as measured by downtime. Two platforms can share a model's capacity, an identity provider, your gateway, or your own bug.
3. **Your own dependencies often matter more.** A second provider doesn't help if your gateway is the single point of failure.

Failover moves load

```
Failover with 1,500,000 ITPM of total load:
  each provider limit 1,000,000 ITPM, normal share 750,000: after failover serves  67%
  each provider limit 2,000,000 ITPM, normal share 750,000: after failover serves 100%
  each provider limit 3,000,000 ITPM, normal share 750,000: after failover serves 100%
```

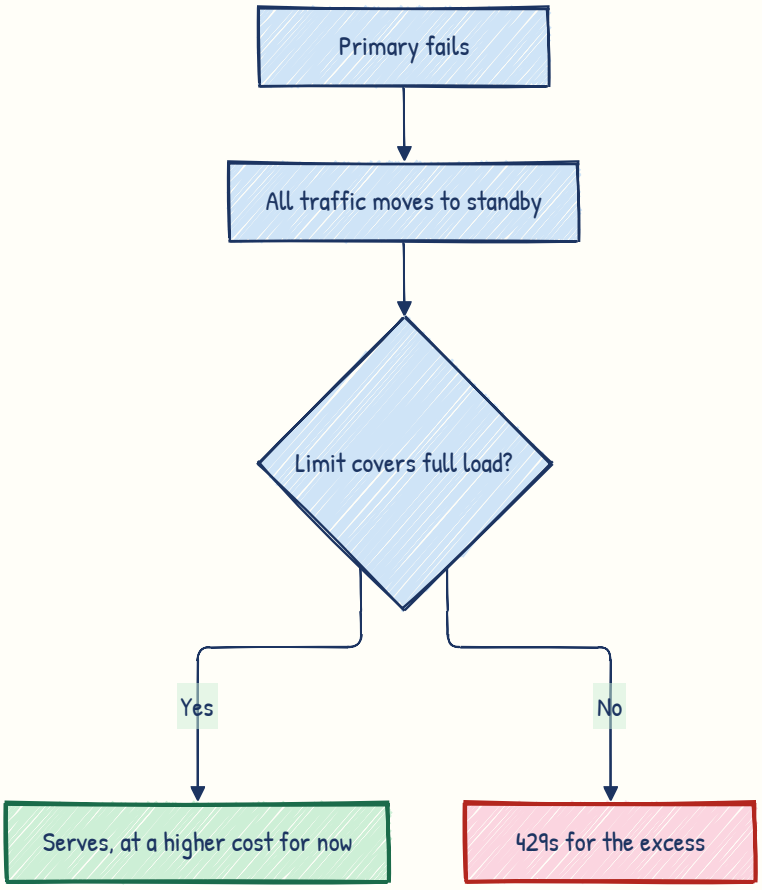


Figure 3. Failover moves all the load to one place. The standby has to have room for it, plus a cold cache.

A standby with too small a limit fails over into a wall of 429s. Other surprises on failover, which I'd list in a runbook:

Surprise	Why	Plan
Cold prompt cache	Caches are isolated	Temporarily higher uncached input, cost and quota use
Acceleration limits	A sudden traffic jump can produce 429s, per the docs	Keep some traffic flowing there always
Different features	Model and platform differ	Evals on that path
Different retention	Features and processors differ	Data review, in advance

Keep a little live traffic on the standby. A path that receives nothing isn't known to work, and acceleration limits make a sudden jump risky.

Disaster recovery

What needs recovering? Standard terms: **RPO** is how much data you can lose, and **RTO** is how long recovery can take.

Asset	If lost	RPO need	Approach
Prompts, configs, evals	Can't release	Zero	Git, replicated
Conversations and agent state	Users lose work	Low	Replicated database
Tool ledger and audit	Compliance and duplicate actions	Zero	Replicated, append-only
Vector index	Retrieval stops	Rebuildable	Keep sources, and time a rebuild
Embeddings	Costly to recompute	Rebuildable	Back up, as rebuilds cost money and time
Caches	Slower, costlier	None	Accept the cold start
Memory	Personalization	Moderate	Replicate

Rebuild time is the catch. Re-embedding a large corpus takes hours, or longer, and the embedding API has its own limits. Measure it. A vector index with "just rebuild it" as its plan may have a recovery time of days.

Idempotency during recovery. After failover, in-flight agent runs are uncertain: did the refund happen? AI08 L3's ledger exists for this, and AI08's rule was that an unfinished call doesn't run again automatically. Resume from a saved state, and have a person review uncertain writes.

Degrade rather than fail. Decide per feature what happens with no model: a static help page, search results with no summary, or queueing the request. A kill switch per feature (AI09 L5) lets you shed quickly.

Rehearse. A runbook that's never run is fiction. Block the primary in staging, then in production at a quiet time.

Why the obvious fixes fall short

Obvious idea	Why it falls short
Treat prompts as config, edited live	No evals, canary or rollback
Pin a model forever	It'll retire
Canary for an hour	Too few outcomes to detect a small drop
One queue, first-come	A flood delays everyone
A second provider means 99.999%	Only if failures are independent
Failover with a small standby limit	429s. Serves 67% in the example
"We can rebuild the index"	Untimed, and possibly days

How Anthropic solved it

Mechanism	What's published	Source
Lifecycle and notice	States, and at least 60 days	Deprecations page
Usage audit	Console export by key and model	Deprecations page
Workspaces	Limits per workspace	Rate limits page
Data residency	`inference_geo`, at a 1.1x price	Pricing and residency docs
Platforms	API, AWS, Google Cloud, Foundry and Bedrock	Docs
Migration guides	Per model, and by parameter	Migration docs

Anthropic's own deployment practices for its services aren't described in what I've read. The stages above are general practice, and AI01 L2 and HLD M13 are their source in this course.

The production architecture

This is a recommended reference design, not a specific company's system.

Baseline: edit the prompt, redeploy, and hope.

Improved: prompts in Git, evals in CI, and canaries. New problem: it's safe for changes you make, and doesn't address tenants' interference, retirements, or outages.

Production: a release pipeline for all artifacts, fair scheduling, and a rehearsed failure path.

Component	Why it exists	How it communicates
Artifact registry	Versions of prompts, models and indexes	Git, plus versioned config
Release controller	Runs stages, and rolls back on signals	CI and flags
Traffic router	Canary by tenant or session	Gateway
Fair scheduler	Per-tenant queues	Gateway
Model calendar	Retirement alerts	Scheduled job and usage export
Standby path	Tested, with limits sized, and a little traffic	Gateway route
Runbooks	Rehearsed actions	Game days

Failures:

Failure	How you'd notice	What to do
Model retired	404s	Should never get here. Alert from the calendar
Canary passes, then quality slips	Sampled evals at scale	Hold at each ramp stage, and keep rollback ready
Canary mixes conversations	Odd behaviour per session	Assign by session or tenant
Noisy tenant	One tenant's queue grows	Limits, and contact them
Standby unusable	Failover drill fails	That's why you drill
Partner platform retires first	Different schedule	Track both calendars
Uncertain writes after failover	Ledger shows unfinished calls	Don't rerun. Review them

Java

The scheduler's core is a loop that gives each tenant one slot per pass:

```
for (Map.Entry<String, Deque<Integer>> entry : queues.entrySet()) {
    if (served < capacity && !entry.getValue().isEmpty()) {
```

That's round robin. A real scheduler weights by plan and costs by tokens.

Sample size comes from the standard two-proportion approximation:

```
return Math.round(Math.pow(1.96 + 0.84, 2) * variance / (drop * drop));
```

The `drop * drop` in the denominator is the important part: detecting half the drop takes four times the data. That's why small regressions need big suites.

Trade-offs

Decision	Benefit	Cost
Shadow	Real inputs with no harm	Pays twice, and no write tools
Long canary	Detects smaller drops	Slower releases
Fair queues	Protects quiet tenants	Slower for big ones, and more state
Second platform	Independent limits, to a degree	Double the testing
Warm standby	Failover works	Ongoing cost
Dedicated orgs	Strong isolation	Billing and operations

When not to do this: a small internal tool can pin a model, run its evals before each change, and accept an outage. Do still put the retirement date on a calendar.

Common mistakes

Editing prompts in production through an admin screen. It bypasses evals and versions, and you can't say what changed.

Common mistakes

Testing failover only on a slide. Cold caches, small limits and untested parameters only appear in a drill.

What to say in an interview

"I treat prompts, models, tools and indexes as release artifacts: evals, then shadow, canary, and ramp, with rollback in one config change and canaries by session. I size canaries by what they can detect. I'd use per-tenant queues with token-based limits, since one queue lets a flood delay everyone. And for recovery, I don't assume provider failures are independent. I size standby limits, keep some traffic on them, and rehearse."

Certification lens

How this lesson maps to CCAR-F

Mostly out of scope. Nearby: TS 3.6 covers Claude Code in CI, with `-p`, `--output-format json` and `--json-schema`. Releases, canaries, tenancy and DR aren't in the guide.

These are original practice questions, not real or leaked.

Q Q1 · Exam style, scenario: Code Review in CI · Medium. A pipeline runs Claude Code on each pull request, and the next step parses its findings into inline comments. Runs hang waiting for input, and parsing often breaks. What's the best fix?

A. Set a `CLAUDE_HEADLESS` environment variable, and parse the text with a regex

B. Add a CLAUDE.md instruction saying never to ask for input

C. Pipe `yes` into the process, and retry on parse errors

D. Run with `-p`, and use `--output-format json` with `--json-schema`

A D (TS 3.6). `-p` is the non-interactive mode, and a schema gives parseable output. The `CLAUDE_HEADLESS` variable in A doesn't exist. B relies on instructions, and C is a workaround that fixes neither problem.

Q Q2 · You have 2,000 labelled outcomes a week on a route at 95% success. How long would a canary need to detect a one-point drop, with half of traffic on the canary?

A About 8,146 per arm, so 16,292 total. At 2,000 a week, that's about eight weeks, which isn't practical. Rely on a large offline eval for small drops, and use the canary to catch large ones and operational problems.

Q Q3 · List four things that may surprise you on failing over to a second platform.

A Any four: a cold prompt cache, raising cost and quota use; a standby limit too small for the load; acceleration limits on a sudden jump; different model parameters or features causing 400s; a different retirement date; different data handling; and untested output quality.

Revision - Lesson 6

→ Prompts, models, tools, indexes and API versions are all release artifacts. Version and log them.

→ Stages: evals, shadow, canary, ramp, with rollback. Shadow can't run write tools.

→ Canary by session or tenant. A one-point drop from 95% takes about 8,000 outcomes per arm.

→ Lifecycle: active, legacy, deprecated, retired. At least 60 days' notice. Retired means failing requests.

→ Partner platforms have their own schedules.

→ One queue lets a flood delay everyone. Use per-tenant limits, queues and token-based shares.

→ Check who processes data per platform. `inference_geo` costs 1.1x, and shares rate limits.

→ Serial dependencies multiply downward. Two providers are only as good as their independence.

→ Failover moves load: size standby limits, expect a cold cache, keep some traffic flowing.

→ Recovery: time your index rebuilds, don't rerun uncertain writes, and rehearse.

Your notes